

1. What is async/await?

- Syntactic sugar over Promises.
 - Makes asynchronous code look synchronous.
 - `async` function always returns a Promise.
 - `await` pauses execution until the Promise resolves or rejects.
-

2. Syntax

```
async function myFunction() {  
  const result = await someAsyncTask();  
  console.log(result);  
}
```

3. Basic Example

```
function fetchData() {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve("Data received!"), 1000);  
  });  
}  
  
async function getData() {  
  console.log("Fetching...");  
  const data = await fetchData();  
  console.log(data);  
}  
  
getData();
```

4. Error Handling

```
function fetchWithError() {  
  return new Promise((_, reject) => setTimeout(() => reject("Error occurred!"),  
    1000));  
}
```

```
}

async function handleError() {
  try {
    const data = await fetchWithError();
    console.log(data);
  } catch (error) {
    console.log("Caught:", error);
  }
}

handleError();
```

5. Using await with Multiple Promises

Sequential Execution

```
async function sequential() {
  const result1 = await Promise.resolve(10);
  const result2 = await Promise.resolve(20);
  console.log(result1, result2);
}
```

Parallel Execution (Faster)

```
async function parallel() {
  const [res1, res2] = await Promise.all([Promise.resolve(10),
  Promise.resolve(20)]);
  console.log(res1, res2);
}
```

6. Async Arrow Functions

```
const fetchData = async () => {
  const data = await Promise.resolve("Arrow async works!");
  console.log(data);
};

fetchData();
```

7. Returning Values

```
async function returnValue() {  
  return 42;  
}  
returnValue().then(value => console.log(value)); // 42
```

8. Top-level await (ES2022+)

```
const data = await Promise.resolve("Top-level await!");  
console.log(data);
```

9. Quick Comparison: Promises vs Async/Await

Feature	Promise	Async/Await
Syntax	<code>.then().catch()</code>	<code>async</code> + <code>await</code>
Readability	Can be chained	Looks synchronous
Error handling	<code>.catch()</code>	<code>try...catch</code>
Multiple tasks	<code>Promise.all()</code>	<code>await Promise.all()</code>

10. Best Practices

1. Use `await` inside `async` functions only.
2. Use `Promise.all` for parallel tasks.
3. Wrap `await` in `try/catch` for error handling.
4. Avoid blocking the main thread with long synchronous code inside `async`.